
Guide complet API MVola Merchant Pay

Configuration → Authentification → Transaction sortante → Webhook
→ Transaction entrante → Détails de transaction

Projet · mvola-prof

API · MVOLA-Merchant-Pay-API 1.0.0 + Authentification

Dernière vérification live · sandbox, 2026-07-28

Environnements · devapi.mvola.mg (sandbox) · api.mvola.mg (production)

OFFICIEL OpenAPI 3.0.1 de MVola et PDF du portail développeur

CAPTURÉ Payload réel observé contre le sandbox — preuve, pas supposition

CODE Ce que cette application envoie réellement (`src/lib/mvola/`)

INDICATIF Forme d'orientation, jamais observée — à ne pas confondre avec une preuve

Table des matières

- | | |
|---|--|
| 1. Vue d'ensemble du flux | 8. Webhook de callback — <code>PUT</code> vers votre URL |
| 2. Configuration | 9. Détails de transaction — <code>GET</code>
<code>/transactionReference</code> |
| 3. Authentification — <code>POST /token</code> | 10. Réconciliation et idempotence |
| 4. En-têtes communs des appels transactionnels | 11. Codes d'erreur |
| 5. Transaction OUT — cash-out (marchand → joueur) | 12. Pièges connus |
| 6. Transaction IN — dépôt (joueur → marchand) | 13. Antisèche curl |
| 7. Polling de statut — <code>GET</code>
<code>/status/{serverCorrelationId}</code> | |
-

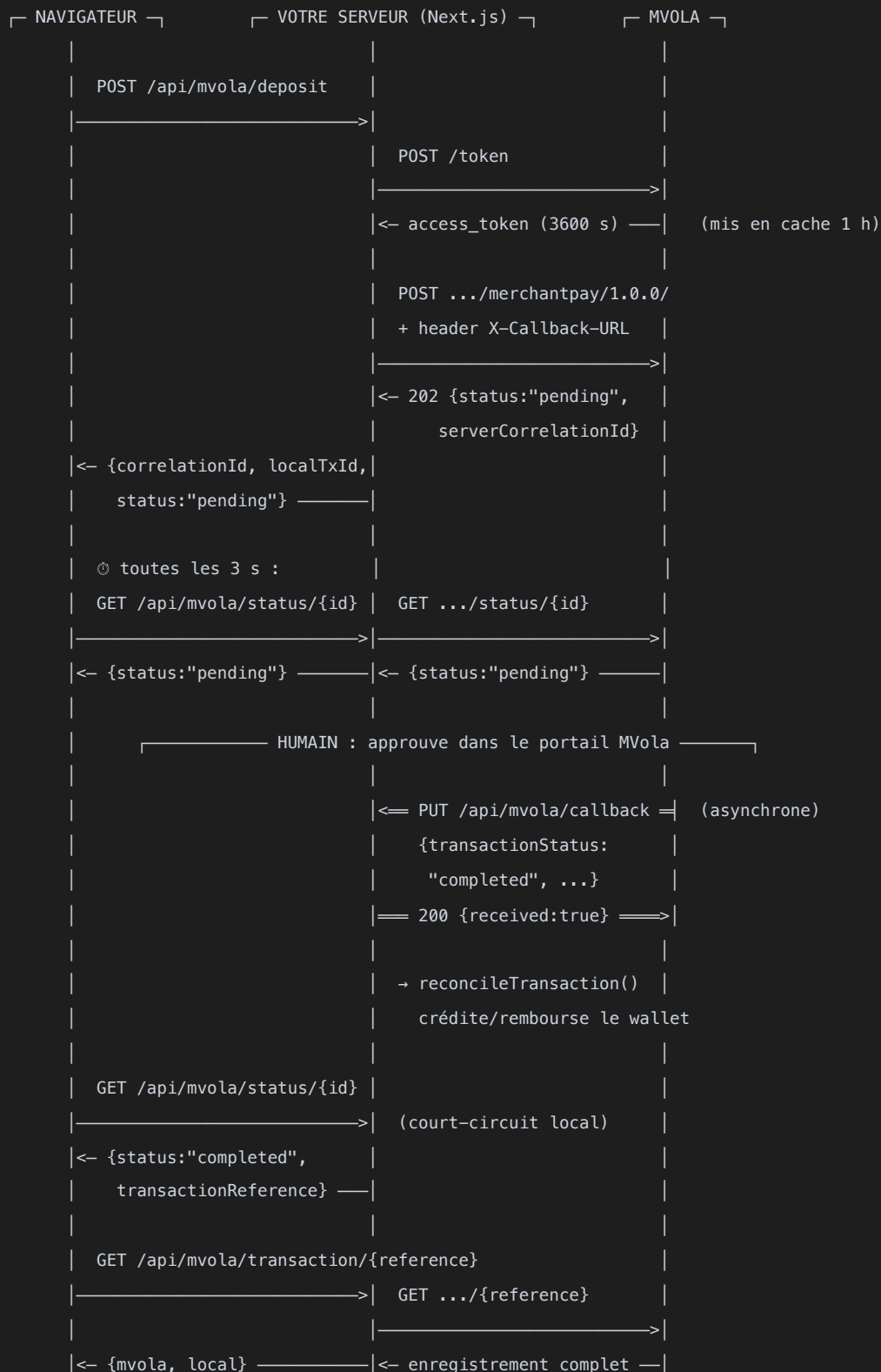
1. Vue d'ensemble du flux

MVola publie **deux API**, **quatre opérations au total** :

API	Opération	Méthode + chemin
Authentication	Jeton OAuth	POST /token
Merchant Pay	Initier une transaction	POST /mvola/mm/transactions/type/merchantpay/1.0.0/
Merchant Pay	Statut d'une transaction	GET /mvola/mm/transactions/type/merchantpay/1.0.0/status/{serverCorrelationId}
Merchant Pay	Détails d'une transaction réglée	GET /mvola/mm/transactions/type/merchantpay/1.0.0/{transactionReference}

Il n'y a **pas** d'endpoint « dépôt » et d'endpoint « retrait » séparés. Le même POST / sert les deux sens : c'est l'orientation debitParty / creditParty qui décide qui paie qui.

Séquence complète



Deux chemins mènent au règlement — callback ET polling. Ils sont redondants volontairement : celui qui arrive en premier règle la transaction, le second devient un no-op grâce à la garde d'idempotence (§ 10). Si votre tunnel webhook tombe, le polling rattrape ; si le navigateur est fermé, le callback rattrape.

Points d'entrée exposés par cette application

Route	Méthode	Rôle
/api/mvola/token	POST	Debug uniquement — renvoie le jeton courant
/api/mvola/deposit	POST	Initie un dépôt (joueur → marchand)
/api/mvola/withdraw	POST	Initie un cash-out (marchand → joueur)
/api/mvola/status/[correlationId]	GET	Proxy du statut + réconciliation
/api/mvola/callback	PUT	Webhook appelé par MVola
/api/mvola/transaction/[transactionReference]	GET	Détails MVola + enregistrement local
/api/config/polling	GET	Livre pollIntervalMs / pollTimeoutMs au navigateur

2. Configuration

2.1 Côté portail MVola (une fois)

Séquence décrite dans `docs/mvola-reference/01-developer-guide.pdf` :

1. Créer un compte sur <https://developer.mvola.mg/devportal/>.
2. Déclarer une **application** (c'est elle qui porte le nom partenaire).
3. **Souscrire** l'application aux deux API : *Authentication* et *MVOLA-Merchant-Pay-API*.
4. Générer les **clés sandbox** → vous obtenez `Consumer Key` et `Consumer Secret`.
5. Un **MSISDN marchand** sandbox vous est attribué.
6. La page **Transaction Approvals** du portail est celle où l'on approuve manuellement chaque transaction sandbox (voir § 2.4).
7. Le passage en production exige la checklist sécurité (`docs/mvola-reference/03-web-security-checklist.pdf`, 9 contrôles web + checklist mobile).

2.2 URLs de base — OFFICIEL

Environnement	Base URL
Sandbox	<code>https://devapi.mvola.mg</code>
Production	<code>https://api.mvola.mg</code>

⚠ Le fichier OpenAPI officiel liste `https://pre-api.mvola.mg/...` dans son bloc `servers`. C'est l'hôte de *pré-production* interne de MVola ; il n'est **pas** celui distribué aux intégrateurs. Utilisez `devapi` / `api` comme ci-dessus.

Dans ce code, **un seul endroit** résout cette URL — `src/lib/mvola/base-url.ts` :

```
// src/lib/mvola/base-url.ts <span class="badge b-cod">CODE</span>
const PRODUCTION_BASE_URL = "https://api.mvola.mg";
const SANDBOX_BASE_URL    = "https://devapi.mvola.mg";

export function getBaseUrl(): string {
  return process.env.MVOLA_ENV === "production"
    ? PRODUCTION_BASE_URL
    : SANDBOX_BASE_URL; // défaut : sandbox, y compris si MVOLA_ENV est vide ou inconnu
}
```

C'est la **règle R2** du projet : `auth.ts` (jeton) et `client.ts` (transactions) importent tous deux ce helper, pour qu'il soit impossible d'obtenir un jeton d'un environnement et de poster les transactions dans l'autre.

2.3 Variables d'environnement

BASH

```
# .env / .env.local — ne jamais committer les valeurs
MVOLA_CONSUMER_KEY=<clé consommateur du portail>
MVOLA_CONSUMER_SECRET=<secret consommateur du portail>
MVOLA_MERCHANT_MSISDN=<MSISDN marchand sandbox>
MVOLA_PARTNER_NAME=<nom sous lequel l'app est enregistrée>
MVOLA_COMPANY_NAME=<cosmétique uniquement>
MVOLA_ENV=sandbox # "production" pour basculer, tout le reste = sandbox
MVOLA_CALLBACK_URL=https://<tunnel>.ngrok-free.app/api/mvola/callback

# Politique de polling côté navigateur (src/lib/mvola/polling.ts)
MVOLA_POLL_INTERVAL_MS=3000 # défaut 3 s
MVOLA_POLL_TIMEOUT_MS=120000 # défaut 120 s
```

Variable	Où elle est lue	Envoyée à MVola comment
MVOLA_CONSUMER_KEY / _SECRET	auth.ts	Authorization: Basic base64(key:secret) sur /token
MVOLA_MERCHANT_MSISDN	client.ts	En-tête UserAccountIdentifier: msisdn;<valeur> et dans debitParty / creditParty
MVOLA_PARTNER_NAME	client.ts	En-tête partnerName et metadata[0] du corps
MVOLA_CALLBACK_URL	client.ts	En-tête X-Callback-URL (uniquement sur le POST d'initiation)
MVOLA_ENV	base-url.ts uniquement	Choisit l'hôte, rien d'autre

MVOLA_POLL_* ne partent jamais vers MVola : ils sont servis au navigateur par GET /api/config/polling, jamais via NEXT_PUBLIC_, pour qu'aucune variable serveur ne fuite (règle R5).

2.4 Sandbox — ce qu'il faut savoir avant de perdre une heure

- **Rien ne se règle tout seul.** Une transaction sandbox reste pending jusqu'à ce qu'un humain l'approuve sur la page *Transaction Approvals* du portail. L'application a l'air figée : elle vous attend.
- **Seuls les numéros de test fonctionnent :** 0343500003 et 0343500004. Le guide officiel est explicite : tout autre numéro ne fonctionnera pas en sandbox.
- **Le callback exige une URL publique.** En local, un tunnel (ngrok) est obligatoire :

BASH

```
ngrok http 3000 --domain=votre-domaine-reserve.ngrok-free.app
```

Un domaine réservé évite que l'URL change à chaque redémarrage — sinon MVOLA_CALLBACK_URL devient silencieusement faux. L'inspecteur ngrok (http://localhost:4040) montre le corps brut de chaque callback, sans ajouter de log dans l'application.

2.5 Préflight

`npm run preflight` (`scripts/preflight.mjs`) vérifie, quelques minutes avant une démo :

1. toutes les variables `MVOLA_*` requises sont non vides,
2. les identifiants sont vivants — un vrai `access_token` peut être obtenu,
3. `MVOLA_CALLBACK_URL` répond **depuis l'extérieur du process** (le tunnel n'a pas expiré).

Un run vert prouve que les identifiants et la connectivité sont bons *maintenant*. Il ne prouve pas qu'une transaction ira jusqu'au règlement.

3. Authentification — POST /token

OAuth 2.0, grant **client_credentials**, scope EXT_INT_MVOLA_SCOPE.

3.1 Requête — OFFICIEL

HTTP

```
POST /token HTTP/1.1
Host: devapi.mvola.mg
Authorization: Basic <base64(consumerKey + ":" + consumerSecret)>
Content-Type: application/x-www-form-urlencoded
Cache-Control: no-cache

grant_type=client_credentials&scope=EXT_INT_MVOLA_SCOPE
```

Implémentation exacte — src/lib/mvola/auth.ts CODE :

TS

```
const credentials = Buffer.from(`${key}:${secret}`).toString("base64");

const response = await fetch(`${getBaseUrl()}/token`, {
  method: "POST",
  headers: {
    Authorization: `Basic ${credentials}`,
    "Content-Type": "application/x-www-form-urlencoded",
  },
  body: "grant_type=client_credentials&scope=EXT_INT_MVOLA_SCOPE",
});
```

Note : le corps est une **chaîne URL-encodée**, pas du JSON. Envoyer {"grant_type":"client_credentials"} en JSON échoue.

3.2 Réponse 200 — OFFICIEL

JSON

```
{
  "access_token": "eyJ4NXQiOiJ0bUp0T0dVeE16WmxZ...",
  "scope": "EXT_INT_MVOLA_SCOPE",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

expires_in est en **secondes** (3600 = 1 h).

3.3 Stratégie de cache

```
// src/lib/mvola/auth.ts CODE
let cachedToken: { access_token: string; expiresAt: number } | null = null;

export async function getToken(): Promise<string> {
  // Rafraîchit dès qu'il reste moins de 60 s de validité
  if (cachedToken && Date.now() < cachedToken.expiresAt - 60_000) {
    return cachedToken.access_token;
  }
  const token = await fetchToken();
  cachedToken = {
    access_token: token.access_token,
    expiresAt: Date.now() + token.expires_in * 1000,
  };
  return cachedToken.access_token;
}
```

- Cache **en mémoire du process** — perdu au redémarrage du serveur (acceptable en PoC ; en multi-instance chaque instance refait son propre jeton).
 - Marge de **60 secondes** avant expiration, pour qu'un jeton ne meure jamais en vol.
 - **Ne jamais logger la valeur du jeton**, ni la clé, ni le secret. Loggez `"Token refreshed, expires in Xs"` et rien d'autre.
-

4. En-têtes communs des appels transactionnels

4.1 Ce que l'OpenAPI officiel exige — OFFICIEL

En-tête	POST /	GET /status/{id}	GET /{ref}
Version	requis	requis	requis
X-CorrelationID	requis	requis	requis
Cache-Control	requis	requis	requis
UserAccountIdentifier	—	requis	requis
partnerName	—	requis	non requis
X-Callback-URL	optionnel	—	—
Accept , Accept-Charset	optionnel	optionnel	optionnel
CellIdA , GeoLocationA , CellIdB , GeoLocationB	optionnel	optionnel	optionnel

Deux observations importantes :

- Le `POST /` ne liste pas `UserAccountIdentifier` ni `partnerName` comme requis dans l'OpenAPI, mais la pratique et la documentation projet les envoient quand même — c'est sans risque et cohérent.
- `GET /{transactionReference}` (détails) est la **seule** opération qui n'exige pas `partnerName`. L'envoyer quand même est inoffensif, ce que fait ce code plutôt que de dupliquer la fonction de construction d'en-têtes.

4.2 Ce que ce code envoie réellement — CODE

```
// src/lib/mvola/client.ts
function buildHeaders(token, callbackUrl?, userAccountMsisdn?) {
  const accountMsisdn = userAccountMsisdn ?? process.env.MVOLA_MERCHANT_MSISDN;
  const headers = {
    Accept: "*/*",
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
    Version: "1.0",
    "X-CorrelationID": crypto.randomUUID(), // NOUVEAU à chaque requête
    UserLanguage: "FR",
    UserAccountIdentifier: `msisdn;${accountMsisdn}`,
    partnerName: process.env.MVOLA_PARTNER_NAME,
    "Cache-Control": "no-cache",
  };
  if (callbackUrl) headers["X-Callback-URL"] = callbackUrl;
  return headers;
}
```

Rendu concret sur le fil :

```
Accept: */*
Content-Type: application/json
Authorization: Bearer eyJ4NXQiOiJ0bUp0T0dVeE16WmxZ...
Version: 1.0
X-CorrelationID: 3f2c9a10-8b41-4d7e-9c02-5f6a1b8e4d33
UserLanguage: FR
UserAccountIdentifier: msisdn;0340000000
partnerName: MonPartenaire
Cache-Control: no-cache
X-Callback-URL: https://mon-tunnel.ngrok-free.app/api/mvola/callback
```

Trois identifiants à ne jamais confondre :

Identifiant	Qui le crée	Portée	Où il apparaît
X-CorrelationID	vous , un UUID par requête HTTP	une requête	en-tête ; ressort dans <code>metadata.XCorrelationId</code>
serverCorrelationId	MVola , à l'initiation	toute la transaction	réponse du <code>POST</code> , clé du polling, présent dans le callback
transactionReference / objectReference	MVola , au règlement	transaction réglée	callback + réponse de statut ; clé de l'endpoint détails

X-CorrelationID et serverCorrelationId sont **différents** — la capture live le confirme (§ 8.3). Ne les traitez jamais comme interchangeables.

5. Transaction OUT — cash-out (marchand → joueur)

L'argent part du compte marchand vers le joueur : `debitParty` = marchand, `creditParty` = joueur.

5.1 Requête complète

HTTP

```
POST /mvola/mm/transactions/type/merchantpay/1.0.0/ HTTP/1.1
Host: devapi.mvola.mg
Authorization: Bearer <access_token>
Content-Type: application/json
Accept: */*
Version: 1.0
X-CorrelationID: 3f2c9a10-8b41-4d7e-9c02-5f6a1b8e4d33
UserLanguage: FR
UserAccountIdentifier: msisdn;0340000000
partnerName: MonPartenaire
Cache-Control: no-cache
X-Callback-URL: https://mon-tunnel.ngrok-free.app/api/mvola/callback

{
  "amount": "1000",
  "currency": "Ar",
  "descriptionText": "Game withdrawal",
  "requestingOrganisationTransactionReference": "9d4e1f77-0c3b-4a52-8e19-2b7c6f0a1d84",
  "originalTransactionReference": "c1a7b3e5-6d24-4f80-9b31-8e0d5c2a7f96",
  "requestDate": "2026-07-28T18:42:11.503Z",
  "debitParty": [{ "key": "msisdn", "value": "0340000000" }],
  "creditParty": [{ "key": "msisdn", "value": "0343500003" }],
  "metadata": [
    { "key": "partnerName", "value": "MonPartenaire" },
    { "key": "fc", "value": "USD" },
    { "key": "amountFc", "value": "1" }
  ]
}
```

5.2 Champs du corps — OFFICIEL (schéma OpenAPI) + CODE

Champ	Type	Obligatoire	Comment il est rempli ici
amount	string	oui	<code>String(amount)</code> — jamais un nombre JSON
currency	string	oui	<code>"Ar"</code>
descriptionText	string	oui	<code>"Game withdrawal"</code> par défaut
requestingOrganisationTransactionReference	string	oui	<code>crypto.randomUUID()</code> — frais à chaque appel
originalTransactionReference	string	oui (schéma)	<code>crypto.randomUUID()</code>
requestDate	string	oui	<code>new Date().toISOString()</code> → ISO-8601 UTC avec millisecondes
debitParty	<code>[[key,value]]</code>	oui	<code>[[key:"msisdn", value: MERCHANT_MSISDN]]</code>
creditParty	<code>[[key,value]]</code>	oui	<code>[[key:"msisdn", value: playerMsisdn]]</code>
metadata	<code>[[key,value]]</code>	oui	<code>partnerName</code> , <code>fc</code> , <code>amountFc</code>

Attention metadata sur le cash-out. Le code envoie `fc: "USD"` et `amountFc: "1"` dans `initiateWithdrawal()`, alors que le dépôt envoie `fc: "Ar"` et `amountFc = montant`. `fc` / `amountFc` désignent la devise étrangère et son montant. La valeur `USD / 1` du cash-out est un reliquat des exemples MVola ; le sandbox l'accepte. Si un contrôle de cohérence de devise apparaît en production, alignez-la sur `"Ar"` / le montant réel.

5.3 Réponse — OFFICIEL

MVola répond **202 Accepted** (la documentation projet mentionne aussi 200 ; le code accepte toute réponse `response.ok`, donc 2xx) :

JSON

```
{
  "status": "pending",
  "serverCorrelationId": "550e8400-e29b-41d4-a716-446655440000",
  "notificationMethod": "callback"
}
```

Seuls `status` et `serverCorrelationId` sont typés côté application (`WithdrawalResponse`) ; `notificationMethod` est documenté par MVola mais non consommé.

`serverCorrelationId` est la seule poignée utilisable ensuite. Conservez-le.

5.4 Le wrapper applicatif — POST /api/mvola/withdraw

Requête :

JSON

```
{ "msisdn": "0343500003", "amount": 1000, "description": "Test cash-out" }
```

(`playerMsisdn` est accepté comme alias historique de `msisdn` . `amount` accepte un nombre ou une chaîne numérique, mais doit être un **entier positif**.)

Réponse 200 :

JSON

```
{
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "localTxId": "b7e2...",
  "status": "pending"
}
```

Ordre des opérations — il est délibéré :

```
1. Valider le corps                                → 400 si invalide
2. debitWallet(msisdn, amount) ← RÉSERVE, AVANT tout await
                                     → 409 si solde insuffisant
                                     (MVola n'est jamais appelé)
3. getToken() puis initiateWithdrawal()
   ├── throw → creditWallet(msisdn, amount) ← REMBOURSEMENT
   │         aucun enregistrement créé      → 502
   └── ok    → createTransaction({ correlationId: <celui de MVola>,
                                   walletSettled: true })
4. Réponse { correlationId, localTxId, status: "pending" }
```

La réserve est **synchrone**, avant le premier `await` : deux cash-outs concurrents ne peuvent pas dépenser le même solde. `walletSettled: true` signifie « le portefeuille a déjà bougé » — c'est ce qui permettra un remboursement si MVola refuse (§ 10).

Code	Corps
400	<code>{"error":"msisdn and amount are required"}</code>
400	<code>{"error":"amount must be a positive integer"}</code>
409	<code>{"error":"Insufficient funds","balance":500,"requested":1000}</code>
502	<code>{"error":"MVola API error","details":"MVola merchant pay endpoint returned 401: ..."}</code>

6. Transaction IN — dépôt (joueur → marchand)

Même endpoint, parties inversées. `debitParty` = joueur, `creditParty` = marchand.

6.1 Requête complète

HTTP

```
POST /mvola/mm/transactions/type/merchantpay/1.0.0/ HTTP/1.1
Host: devapi.mvola.mg
Authorization: Bearer <access_token>
Content-Type: application/json
Accept: */*
Version: 1.0
X-CorrelationID: 7b1d4e2f-3a65-49c8-b0d7-1e5f9c8a2b40
UserLanguage: FR
UserAccountIdentifier: msisdn;0340000000
partnerName: MonPartenaire
Cache-Control: no-cache
X-Callback-URL: https://mon-tunnel.ngrok-free.app/api/mvola/callback

{
  "amount": "5000",
  "currency": "Ar",
  "descriptionText": "Game deposit",
  "requestingOrganisationTransactionReference": "e4f8a2c1-9b70-4d63-85ae-3c1b7f0d2e59",
  "originalTransactionReference": "a2d5c7b9-4e18-4a30-9f62-7b0c3e5d1a84",
  "requestDate": "2026-07-28T18:55:03.117Z",
  "debitParty": [{ "key": "msisdn", "value": "0343500003" }],
  "creditParty": [{ "key": "msisdn", "value": "0340000000" }],
  "metadata": [
    { "key": "partnerName", "value": "MonPartenaire" },
    { "key": "fc", "value": "Ar" },
    { "key": "amountFc", "value": "5000" }
  ]
}
```

Différences avec le cash-out — **exactement quatre** :

	Cash-out	Dépôt
<code>debitParty</code>	marchand	joueur
<code>creditParty</code>	joueur	marchand
<code>descriptionText</code> défaut	"Game withdrawal"	"Game deposit"
<code>metadata.fc</code> / <code>amountFc</code>	"USD" / "1"	"Ar" / le montant

6.2 Réponse — identique au cash-out

JSON

```
{ "status": "pending", "serverCorrelationId": "550e8400-e29b-41d4-a716-446655440000" }
```

6.3 Le wrapper applicatif — POST /api/mvola/deposit

Requête : { "msisdn": "0343500003", "amount": 5000 }

Réponse 200 : { "correlationId": "...", "localTxId": "...", "status": "pending" }

Différence structurelle capitale avec le cash-out : le portefeuille n'est PAS crédité ici. L'enregistrement est créé avec `walletSettled: false`, et le crédit n'a lieu qu'au règlement confirmé par MVola (règle R3 : « un solde ne change que quand MVola confirme »).

```
1. Valider le corps → 400 si invalide
2. getToken() puis initiateDeposit()
   ├── throw → 502, AUCUN enregistrement créé
   └── ok → createTransaction({ correlationId: <MVola>, walletSettled: false })
3. Réponse { correlationId, localTxId, status: "pending" }
```

Asymétrie voulue :

	Dépôt	Cash-out
Le portefeuille bouge...	au règlement	à la demande (réserve)
<code>walletSettled</code> initial	<code>false</code>	<code>true</code>
Si MVola échoue à l'initiation	rien à défaire	remboursement immédiat
Si MVola répond <code>failed</code> plus tard	marquer <code>failed</code> , aucun mouvement	rembourser

7. Polling de statut — `GET /status/{serverCorrelationId}`

7.1 Appel à MVola

```
GET /mvola/mm/transactions/type/merchantpay/1.0.0/status/550e8400-e29b-41d4-a716-446655440000
HTTP/1.1
Host: devapi.mvola.mg
Authorization: Bearer <access_token>
Version: 1.0
X-CorrelationID: <nouvel UUID>
UserAccountIdentifier: msisdn;0340000000
partnerName: MonPartenaire
Cache-Control: no-cache
```

`partnerName` est **requis** sur cette opération. `X-Callback-URL` n'est pas envoyé (`buildHeaders(token)` sans second argument).

7.2 Réponse — **VÉRIFIÉ** contre le sandbox le 2026-07-28

```
{
  "status": "completed",
  "serverCorrelationId": "550e8400-e29b-41d4-a716-446655440000",
  "notificationMethod": "callback",
  "objectReference": "1234567"
}
```

🔴 **Le piège central de cette API.** La réponse de statut nomme son champ de progression `status`, et sa référence `objectReference`. Elle n'utilise **pas** `transactionStatus` / `transactionReference` — cette orthographe-là est celle du *callback* et de l'endpoint *détails*. Une intégration qui lit `transactionStatus` sur la réponse de statut ne trouve rien, et laisse chaque transaction sans état.

Valeurs de `status` : `pending` | `completed` | `failed`.

7.3 Le lecteur unifié — `parseMvolaStatus()`

Parce que les deux orthographes coexistent réellement selon l'endpoint, un lecteur unique les accepte toutes les deux :

TS

```
// src/lib/mvola/status.ts <span class="badge b-cod">CODE</span>
export function parseMvolaStatus(payload: unknown):
  { status: TransactionStatus; reference?: string } {

  const record = asPlainRecord(payload); // null/undefined/tableau/primitive → {}

  const rawStatus =
    typeof record.status === "string" ? record.status : record.transactionStatus;
  const status = isKnownStatus(rawStatus) ? rawStatus : "pending"; // ← jamais terminal

  const rawReference = record.objectReference ?? record.transactionReference;
  const reference =
    typeof rawReference === "string" && rawReference !== "" ? rawReference : undefined;

  return { status, reference };
}
```

Deux propriétés de sûreté :

- **Un statut illisible vaut `pending` , jamais un état terminal.** Un payload corrompu ne peut donc jamais régler (ni faire échouer) une transaction par accident.
- **Aucune exception.** `null` , un tableau, une chaîne, un nombre : tout est toléré.

La route de statut **et** la route de callback passent par ce même lecteur — c'est la règle R4 : les deux points d'entrée ne peuvent pas être en désaccord sur ce que MVola a dit.

7.4 Le wrapper applicatif — `GET /api/mvola/status/[correlationId]`

Réponse 200 — les deux orthographes sont renvoyées au navigateur, pour que l'UI existante continue de fonctionner :

JSON

```
{
  "status": "completed",
  "serverCorrelationId": "550e8400-e29b-41d4-a716-446655440000",
  "objectReference": "1234567",
  "transactionStatus": "completed",
  "transactionReference": "1234567"
}
```

Court-circuit local : si l'enregistrement local est déjà terminal, la route répond sans appeler MVola. C'est une optimisation pure — cet état terminal n'a pu être atteint que parce que MVola l'a dit (règle R1).

Effet de bord : au premier passage terminal, la route appelle `reconcileTransaction()` (§ 10). Un `correlationId` inconnu localement est toléré : la réponse MVola est quand même transmise.

502 `{"error": "..."}` si le jeton ou l'appel MVola échoue.

7.5 Politique de polling côté navigateur

```
// src/lib/mvola/polling.ts <span class="badge b-cod">CODE</span>  
export const POLL_INTERVAL_MS = readMs("MVOLA_POLL_INTERVAL_MS", 3000);  
export const POLL_TIMEOUT_MS = readMs("MVOLA_POLL_TIMEOUT_MS", 120000);
```

TS

Le navigateur les récupère via `GET /api/config/polling` — **jamais** en lisant `polling.ts` (module serveur) ni une variable `NEXT_PUBLIC_`. Si ce fetch échoue, les valeurs par défaut (3 s / 120 s) sont utilisées : le polling ne doit jamais être bloqué par la config.

Le plafond de 120 s est une frontière de *reporting*, pas un échec. Quand il est atteint, l'UI affiche « toujours en attente » — elle ne marque **jamais** `failed`, ne rafraîchit pas le solde, et ne touche pas au portefeuille. La transaction peut encore se régler par callback ensuite (règle R3).

Ne descendez pas sous 3 secondes : risque de rate-limiting côté MVola.

8. Webhook de callback — `PUT` vers votre URL

8.1 Comment MVola sait où appeler

Par l'en-tête `X-Callback-URL` envoyé sur le `POST` d'initiation. Il est optionnel : sans lui, aucun callback n'est livré et seul le polling reste. Il n'est pas envoyé sur les `GET`.

L'URL doit être **publiquement joignable en HTTPS**. En développement, un tunnel est indispensable.

8.2 La méthode est `PUT`, pas `POST`

C'est inhabituel et c'est une source d'erreur classique : un handler déclaré en `POST` ne recevra jamais rien. Dans Next.js App Router :

```
export async function PUT(req: NextRequest): Promise<NextResponse> { ... }
```

TS

8.3 Payload réel — `CAPTURÉ` le 2026-07-28, sandbox, cash-out réglé

Capturé via l'inspecteur ngrok sur une vraie livraison. MSISDN et montants rédigés ; les **noms de champs sont verbatim** :

```
{
  "transactionStatus": "completed",
  "serverCorrelationId": "<uuid>",
  "transactionReference": "<chaîne numérique, ~7 chiffres>",
  "requestDate": "<ISO-8601 avec millisecondes et Z>",
  "debitParty": [{ "key": "msisdn", "value": "<rédigé>" }],
  "creditParty": [{ "key": "msisdn", "value": "<rédigé>" }],
  "fees": [{ "feeAmount": "<rédigé>" }],
  "metadata": [{ "key": "XCorrelationId", "value": "<uuid>" }]
}
```

JSON

Faits établis par cette capture :

- Le callback utilise `transactionStatus` et `transactionReference` — l'orthographe opposée à celle de la réponse de statut. `status` / `objectReference` n'apparaissent pas.
- `transactionStatus` valait `completed` sur un règlement réussi.
- Il n'y a ni `amount` ni `currency` au niveau racine.** Le montant n'est pas renvoyé du tout par le callback. Si vous en avez besoin, prenez-le de votre enregistrement local ou de l'endpoint détails.
- `fees[] . feeAmount` porte les frais MVola.
- `metadata.XCorrelationId` est **différent** de `serverCorrelationId` — c'est l'écho du `X-CorrelationID` que vous aviez envoyé sur la requête d'initiation.

INDICATIF**— à ne pas confondre avec la capture.**

La forme suivante, avec `amount`, `currency` et `MVL-...` comme référence, circule dans plusieurs documents du projet. Elle **n'a jamais été observée** : la vraie référence est une chaîne numérique et le montant est absent.

JSON

```
{ "transactionStatus": "completed", "serverCorrelationId": "...",  
  "transactionReference": "MVL-2026-04-16-001", "amount": "5000", "currency": "Ar", ... }
```

Pourquoi le fallback de `parseMvolaStatus()` reste en place. Une seule livraison, une seule valeur de statut (`completed`), un seul sens (cash-out) ont été observés. Et l'endpoint de *statut* utilise réellement l'autre orthographe. Le fallback fait donc un vrai travail. Il ne pourra être retiré qu'après avoir observé aussi un callback de transaction **échouée** et un callback de **dépôt**.

8.4 Ce que votre handler DOIT faire

Règle	Pourquoi
Toujours répondre <code>200</code>	Tout autre code fait retenter MVola, potentiellement en boucle
Être idempotent	Le même <code>serverCorrelationId</code> peut arriver plusieurs fois
Ne jamais throw	Une exception non attrapée = 500 = retry
Ne pas logger de données personnelles	Règle R5 : pas de MSISDN dans les logs

Implémentation — `src/app/api/mvola/callback/route.ts` **CODE** :

```
export async function PUT(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json();
    const { serverCorrelationId } = body ?? {};

    if (!serverCorrelationId) {
      console.warn("[mvola/callback] Missing serverCorrelationId in payload", body);
      return NextResponse.json({ received: true }, { status: 200 }); // 200 quand même
    }

    const record = getTransactionByCorrelationId(serverCorrelationId);
    if (!record) {
      console.warn("[mvola/callback] Unknown correlationId", serverCorrelationId);
      return NextResponse.json({ received: true }, { status: 200 }); // 200 quand même
    }

    try {
      const { status, reference } = parseMvolaStatus(body); // même lecteur que le polling
      reconcileTransaction(record, status, reference);
    } catch (reconcileErr) {
      console.error("[mvola/callback] Reconciliation error", serverCorrelationId, reconcileErr);
    }

    return NextResponse.json({ received: true }, { status: 200 });
  } catch (err) {
    console.error("[mvola/callback] Unhandled error parsing body", err);
    return NextResponse.json({ received: true }, { status: 200 }); // 200 quand même
  }
}
```

Réponse, dans tous les cas : `200 { "received": true }` — corrélation inconnue, JSON malformé, erreur de réconciliation comprises.

Noter que `serverCorrelationId` est lu **directement** sur le corps : il ne fait pas partie du contrat de `parseMvolaStatus()`, qui ne s'occupe que du statut et de la référence.

⚠ Il y a actuellement un `console.log(body)` non committé en tête de ce handler (visible dans `git diff`). Il a servi à capturer le payload live. Il logge le corps **complet**, MSISDN inclus — donc en contradiction avec la politique « pas de données personnelles dans les logs ». À retirer avant tout commit ou déploiement.

8.5 Sécurité — ce qui n'existe pas

MVola **ne définit aucune signature de webhook**. N'importe qui connaissant votre URL de callback peut poster un corps. Pour la production, envisagez :

- une allowlist d'IP MVola,
- un secret partagé dans le chemin de l'URL de callback,

- ne jamais faire confiance au montant du callback (il n'y est d'ailleurs pas) — se fier à l'enregistrement local, indexé par `serverCorrelationId`.
-

9. Détails de transaction — GET /{transactionReference}

La quatrième et dernière opération. Elle renvoie l'enregistrement **faisant autorité** chez MVola pour une transaction réglée.

9.1 Appel à MVola

HTTP

```
GET /mvola/mm/transactions/type/merchantpay/1.0.0/1234567 HTTP/1.1
Host: devapi.mvola.mg
Authorization: Bearer <access_token>
Version: 1.0
X-CorrelationID: <nouvel UUID>
UserAccountIdentifier: msisdn;0340000000
Cache-Control: no-cache
```

- La référence est **URL-encodée** dans le chemin (`encodeURIComponent`).
- Elle doit être une **référence de règlement** — un `serverCorrelationId` est rejeté.
- `partnerName` n'est pas requis ici (seul endpoint dans ce cas).

9.2 Réponse — [CAPTURÉ, cash-out sandbox réglé]

JSON

```
{
  "amount": "<rédigé>",
  "currency": "Ar",
  "requestDate": "<ISO-8601>",
  "debitParty": [{ "key": "msisdn", "value": "<rédigé>" }],
  "creditParty": [{ "key": "msisdn", "value": "<rédigé>" }],
  "fees": [{ "feeAmount": "<rédigé>" }],
  "metadata": [
    { "key": "originalTransactionResult", "value": "0" },
    { "key": "originalTransactionResultDesc", "value": "0" },
    { "key": "XCorrelationId", "value": "<uuid>" }
  ],
  "transactionStatus": "completed",
  "creationDate": "<ISO-8601>",
  "transactionReference": "<chaîne numérique>"
}
```

Faits établis :

- Orthographe `transactionStatus` / `transactionReference` — cohérente avec le **callback**, et non avec la réponse de statut.
- **Deux horodatages distincts** : `creationDate` et `requestDate` . Sur l'enregistrement observé, `creationDate` précède `requestDate` d'environ 8 minutes. Ne supposez pas que l'un est « l'heure de la transaction » sans savoir lequel vous voulez.

- `metadata` porte `originalTransactionResult` et `originalTransactionResultDesc`, tous deux à `"0"` en cas de succès. Ils **n'apparaissent pas** dans le callback.

Réserve honnête : cette capture provient d'un **cash-out**, pas d'un dépôt. Le jeu de clés d'un dépôt n'a pas été observé et peut différer — au minimum l'orientation `debitParty` / `creditParty` est inversée.

Le type TypeScript reflète cette incertitude : chaque champ nommé est optionnel, et une **index signature** `[key: string]: unknown` préserve toute clé non nommée plutôt que de la perdre.

9.3 Le wrapper applicatif — `GET /api/mvola/transaction/[transactionReference]`

Il sert côte à côte l'enregistrement MVola et l'enregistrement local :

JSON

```
{
  "mvola": { /* transmis VERBATIM, non remodelé */,
  "local": {
    "localTxId": "<uuid>",
    "correlationId": "<uuid>",
    "msisdn": "0343500003",
    "direction": "withdraw",
    "amount": 1000,
    "status": "completed",
    "walletSettled": true,
    "createdAt": 1785000000000,
    "updatedAt": 1785000123000,
    "mvolaReference": "1234567"
  }
}
```

La comparaison est présentée, pas assertée : aucun champ `match` / `verdict` / `diff`. Remodeler `mvola` pour qu'il ressemble à `local` détruirait précisément l'intérêt de montrer deux comptabilités tenues indépendamment.

L'ordre des vérifications compte : le local est cherché en premier. Un 404 ne dépend donc pas de la disponibilité de MVola, et aucun jeton n'est acquis pour une référence inconnue.

Code	Corps
404	<code>{"error":"No local transaction carries that reference"}</code> — MVola n'est pas appelé
502	<code>{"error":"MVola API error","details":"..."}</code> — rien n'est fabriqué pour combler le trou

10. Réconciliation et idempotence

`src/lib/mvola/reconcile.ts` est le point de convergence : le **callback** et le **polling** appellent tous deux cette **fonction**, celui qui arrive en premier règle, le second ne fait rien.

10.1 Table de vérité

direction	walletSettled	statut courant	nouveau statut	action portefeuille	walletSettled après	statut après
deposit	false	pending	completed	crédit +montant	true	completed
deposit	false	pending	failed	aucune	true	failed
withdraw	true	pending	completed	aucune (déjà débité)	true	completed
withdraw	true	pending	failed	remboursement +montant	false	failed
toute ligne	—	non pending	—	no-op (idempotent)	—	inchangé
toute ligne	—	—	pending	no-op (garde 1)	—	inchangé

10.2 Les deux gardes

```
export function reconcileTransaction(record, newStatus, mvolaReference?) {  
  // Garde 1 – un polling intermédiaire ("pending") n'est pas un événement  
  if (newStatus !== "completed" && newStatus !== "failed") return;  
  
  // Relecture AUTORITAIRE depuis le store : l'objet du caller peut être un  
  // instantané pris avant un await  
  const current = getTransactionById(record.localTxId);  
  if (current === undefined) return;  
  
  // Garde 2 – idempotence : une fois réconcilié, tout appel suivant est un no-op  
  if (current.status !== "pending") return;  
  ...  
}
```

Le détail subtil, et il est essentiel. Le `record` passé par l'appelant est traité comme un **identifiant seulement** ; tout est re-lu depuis le store. Le store est *copy-on-write* (`updateTransactionStatus` écrit un **nouvel** objet plutôt que de muter en place), donc un appelant ayant capturé l'enregistrement avant un `await` détient un instantané figé à `status: "pending"` — même si un autre point d'entrée a déjà réglé la transaction entre temps. Faire confiance à cet instantané permettrait de créditer (ou rembourser) deux fois.

C'est exactement le scénario course callback/polling : les deux arrivent, les deux voient `pending` dans leur instantané, un seul le voit dans le store.

10.3 Rétention de la référence

`mvolaReference` est persisté sur l'enregistrement **au moment de la mutation**, depuis `parseMvolaStatus(...).reference`. C'est la règle R7 : sans elle, la transaction réglée devient irrécupérable côté MVola (l'endpoint détails n'a plus de clé). Une transaction réglée sans `mvolaReference` ne peut pas être ouverte — et l'UI le dit, au lieu de fabriquer un enregistrement de remplacement.

11. Codes d'erreur

11.1 MVola → vous — OFFICIEL

Code	Signification	Traitement recommandé
200 / 202	Succès	—
400	Bad Request — paramètres invalides	400 avec le détail. Vérifier : <code>amount</code> en string, MSISDN de test, MSISDN marchand enregistré
401	Unauthorized — jeton expiré ou invalide	Rafraîchir le jeton, retenter une fois ; si toujours 401 → 502
402	Request Failed — la transaction a échoué	502 avec <code>{status:"failed"}</code>
403	Forbidden	502
404	Not Found	404
409	Conflict — <code>requestingOrganisationTransactionReference</code> dupliqué	Générer un nouvel UUID et retenter

11.2 Votre API → le navigateur

Route	Code	Cause
<code>POST /api/mvola/deposit</code>	400	<code>msisdn</code> / <code>amount</code> manquant, ou montant non entier positif
	502	Échec jeton ou appel MVola — aucun enregistrement créé
<code>POST /api/mvola/withdraw</code>	400	idem
	409	Solde insuffisant — MVola n'est pas appelé
	502	Échec MVola — réserve remboursée, aucun enregistrement créé
<code>GET /api/mvola/status/[id]</code>	502	Échec jeton ou appel MVola
<code>GET /api/mvola/transaction/[ref]</code>	404	Aucun enregistrement local ne porte cette référence
	502	Échec jeton ou appel détails MVola
<code>PUT /api/mvola/callback</code>	jamais autre chose que 200	par conception

11.3 Format des erreurs remontées par le client

```
MVola <contexte> returned <status>: <corps brut>
```

Contextes : `merchant pay endpoint`, `deposit merchant pay endpoint`, `transaction status endpoint`, `transaction details endpoint`, `token endpoint`. Cette chaîne finit dans le champ `details` des réponses 502.

12. Pièges connus

12.1 Les orthographes divergent selon l'endpoint — le piège n°1

Endpoint	Champ statut	Champ référence	Source
GET /status/{id}	status	objectReference	VÉRIFIÉ sandbox 2026-07-28
PUT callback	transactionStatus	transactionReference	CAPTURÉ live 09-14
GET /{reference}	transactionStatus	transactionReference	CAPTURÉ live 09-14

Lire une seule orthographe partout laisse la moitié de vos transactions sans statut. Passez **toujours** par `parseMvolaStatus()`.

12.2 Sens débit/crédit inversé

Pour un **payout** vers le joueur : `debitParty` = marchand, `creditParty` = joueur. Les inverser transforme le payout en encaissement — l'argent part dans le mauvais sens.

12.3 `requestingOrganisationTransactionReference` réutilisé

Une valeur en dur → `409 Conflict`. Un `crypto.randomUUID()` frais à chaque appel.

12.4 `amount` envoyé en nombre

`{"amount": 5000}` → rejet. MVola exige une **chaîne** : `{"amount": "5000"}`. Ici, `String(amount)` est appliqué au bord `client.ts`, jamais ailleurs.

12.5 Handler de callback déclaré en `POST`

MVola fait un `PUT`. Un handler `POST` ne recevra rien, sans erreur visible.

12.6 Handler de callback qui renvoie 4xx/5xx

MVola considère cela comme un échec de livraison et retente — potentiellement en boucle. Attrapez tout, répondez 200.

12.7 `Cache-Control: no-cache` oublié

Certains sauts de passerelle MVola mettent les réponses en cache de façon inattendue. L'en-tête est d'ailleurs **requis** par l'OpenAPI sur les trois opérations Merchant Pay.

12.8 Logger le jeton, la clé ou le secret

Un Bearer donne un accès marchand complet jusqu'à expiration. Ne loggez jamais `headers` en entier — il contient l'`Authorization`.

12.9 Attendre qu'une transaction sandbox se règle seule

Elle ne le fera pas. Il faut cliquer *Approve* dans le portail. Si l'app semble bloquée aux étapes d'approbation, elle vous attend.

12.10 Croire que `X-CorrelationID` == `serverCorrelationId`

Non. Le premier est votre UUID par requête HTTP (il ressort dans `metadata.XCorrelationId`), le second est l'identifiant MVola de la transaction.

12.11 Attendre `amount` dans le callback

Il n'y est pas — confirmé par la capture live. Prenez-le du local ou de l'endpoint détails.

13. Antisèche curl

Contre MVola directement

BASH

```
BASE=https://devapi.mvola.mg
KEY=<consumer key>; SECRET=<consumer secret>
MERCHANT=0340000000; PLAYER=0343500003; PARTNER=MonPartenaire
CB=https://mon-tunnel.ngrok-free.app/api/mvola/callback

# 1. Jeton
TOKEN=$(curl -s -X POST "$BASE/token" \
  -H "Authorization: Basic $(printf '%s:%s' "$KEY" "$SECRET" | base64)" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Cache-Control: no-cache" \
  -d 'grant_type=client_credentials&scope=EXT_INT_MVOLA_SCOPE' | jq -r .access_token)

# 2. Initier un dépôt (joueur → marchand)
curl -s -X POST "$BASE/mvola/mm/transactions/type/merchantpay/1.0.0/" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Version: 1.0" \
  -H "X-CorrelationID: $(uuidgen)" \
  -H "UserLanguage: FR" \
  -H "UserAccountIdentifier: msisd;$MERCHANT" \
  -H "partnerName: $PARTNER" \
  -H "Cache-Control: no-cache" \
  -H "X-Callback-URL: $CB" \
  -d "{
    \"amount\": \"5000\",
    \"currency\": \"Ar\",
    \"descriptionText\": \"Game deposit\",
    \"requestingOrganisationTransactionReference\": \"$(uuidgen)\",
    \"originalTransactionReference\": \"$(uuidgen)\",
    \"requestDate\": \"$(date -u +%Y-%m-%dT%H:%M:%S.000Z)\",
    \"debitParty\": [{\"key\": \"msisd\", \"value\": \"$PLAYER\"}],
    \"creditParty\": [{\"key\": \"msisd\", \"value\": \"$MERCHANT\"}],
    \"metadata\": [
      {\"key\": \"partnerName\", \"value\": \"$PARTNER\"},
      {\"key\": \"fc\", \"value\": \"Ar\"},
      {\"key\": \"amountFc\", \"value\": \"5000\"}
    ]
  }"
# → {"status": "pending", "serverCorrelationId": "..."}

# 3. Statut (partnerName REQUIS)
SCID=<serverCorrelationId>
curl -s "$BASE/mvola/mm/transactions/type/merchantpay/1.0.0/status/$SCID" \
```

```

-H "Authorization: Bearer $TOKEN" \
-H "Version: 1.0" -H "X-CorrelationID: $(uuidgen)" \
-H "UserAccountIdentifier: msisd;$MERCHANT" \
-H "partnerName: $PARTNER" -H "Cache-Control: no-cache"
# → {"status":"...", "serverCorrelationId":"...", "objectReference":"..."}

# 4. Détails (référence de RÈGLEMENT, pas le correlationId ; partnerName non requis)
REF=<objectReference>
curl -s "$BASE/mvola/mm/transactions/type/merchantpay/1.0.0/$REF" \
-H "Authorization: Bearer $TOKEN" \
-H "Version: 1.0" -H "X-CorrelationID: $(uuidgen)" \
-H "UserAccountIdentifier: msisd;$MERCHANT" -H "Cache-Control: no-cache"

```

Contre l'API locale

BASH

```

# Jeton (debug)
curl -X POST http://localhost:3000/api/mvola/token

# Dépôt
curl -X POST http://localhost:3000/api/mvola/deposit \
-H "Content-Type: application/json" \
-d '{"msisd":"0343500003","amount":"5000"}'

# Cash-out
curl -X POST http://localhost:3000/api/mvola/withdraw \
-H "Content-Type: application/json" \
-d '{"msisd":"0343500003","amount":"1000","description":"Test cash-out"}'

# Statut
curl http://localhost:3000/api/mvola/status/<correlationId>

# Détails (référence d'une transaction réglée)
curl http://localhost:3000/api/mvola/transaction/<mvolaReference>

# Simuler un callback – DÉVELOPPEMENT UNIQUEMENT
curl -X PUT http://localhost:3000/api/mvola/callback \
-H "Content-Type: application/json" \
-d '{"transactionStatus":"completed","serverCorrelationId":"<scid>","transactionReference":"1234567"}'

```

Un callback simulé ne doit **jamais** tenir lieu de MVola pendant une démonstration : rien de présenté comme une transaction MVola ne peut être produit localement (règle R1).

Annexe — carte des fichiers

Fichier	Rôle
src/lib/mvola/base-url.ts	Seul endroit qui lit <code>MVOLA_ENV</code> (règle R2)
src/lib/mvola/auth.ts	Jeton OAuth + cache mémoire avec marge de 60 s
src/lib/mvola/client.ts	Les 4 appels HTTP MVola + construction des en-têtes
src/lib/mvola/status.ts	<code>parseMvolaStatus()</code> — lecteur unique des deux orthographes
src/lib/mvola/reconcile.ts	Table de vérité MVola → portefeuille, idempotente
src/lib/mvola/polling.ts	<code>POLL_INTERVAL_MS</code> / <code>POLL_TIMEOUT_MS</code>
src/lib/mvola/types.ts	Toutes les formes de payload
src/app/api/mvola/*/route.ts	Les routes Next.js listées en § 1
scripts/preflight.mjs	Contrôle avant démo : env + identifiants + tunnel
docs/mvola-reference/	PDF officiels + OpenAPI, récupérés du portail le 2026-07-28
docs/architecture/_shared.md	Formats de message partagés + capture du callback
docs/architecture/dev-guide.md	Runbook opérateur du walkthrough sandbox

Rafraîchir la documentation officielle

L'API du portail est publique et non authentifiée :

BASH

```
BASE=https://developer.mvola.mg/api/am/devportal/v2
API=5b6887bb-a923-4cb5-84f8-f173f9408792

curl -s "$BASE/apis?limit=100"           # APIs publiées
curl -s "$BASE/apis/$API/swagger"       # définition OpenAPI
curl -s "$BASE/apis/$API/documents?limit=50" # documents attachés
curl -s "$BASE/apis/$API/documents/<documentId>/content" -o out.pdf
```

Ce qui reste non vérifié

Pour rester honnête sur les limites des preuves rassemblées :

1. **Le jeu de clés des détails pour un dépôt** — la capture provient d'un cash-out. L'orientation `debitParty` / `creditParty` diffère au minimum.
2. **Un callback de transaction échouée** — seul `completed` a été observé. La valeur exacte du champ statut sur un échec (`failed` ? autre chose ?) reste supposée.
3. **Un callback de dépôt** — seul un cash-out a été capturé.
4. **Le comportement de retry de MVola** sur un callback non-200 — la politique (nombre de tentatives, backoff) n'est documentée nulle part.

Tant que ces quatre points ne sont pas observés, le fallback double-orthographe de `parseMvolaStatus()` reste nécessaire et ne doit pas être retiré.